

Documentación complementaria

Arquitectura de Odoo (MVC)

Odoo utiliza una arquitectura **Modelo-Vista-Controlador**, pero con esteroides:

Las tablas de la base de datos se definen como clases de Python. El programador no suele escribir SQL; usa el **ORM (Object-Relational Mapping)** de Odoo para interactuar con los datos. Las vistas, se manejan con XML y al declarar los campos se sirve la información, el controlador suele ser el ORM de odoo, una vez cumplas con su estructura el se encarga de la conexión con la base de datos.

Métodos y uso del ORM

Para entender el uso adecuado del ORM en Odoo 18, es vital recordar que estos métodos no solo mueven datos a la base de datos, sino que disparan una cadena de eventos (como el recálculo de campos, el envío de correos o la escritura en el muro de mensajes/chat).

1. El método create(): Se usa para insertar nuevos registros. Siempre recibe una lista de diccionarios (en Odoo 18 se recomienda manejarlo así para soportar creaciones masivas).

Ejemplo: Registrar un nuevo curso desde un método personalizado.

Python

@api.model

def registrar_curso_nuevo(self):

Definimos los valores iniciales

vals = {

'name': 'Guitarra Eléctrica I',

'level': 'basic',

'description': 'Curso introductorio para principiantes',

}

El método create devuelve el 'recordset' del nuevo registro

nuevo_curso = self.create(vals)

return nuevo_curso

2. El método write(): Se usa para actualizar registros existentes. Recibe un diccionario con los campos a cambiar. Se aplica sobre un recordset (uno o varios registros).

Ejemplo: Aprobar a todos los estudiantes de una lista y asignarles una nota mínima.

Python

```
def aprobar_masivamente(self):  
    # 'self' puede contener múltiples registros seleccionados en la vista lista  
    # Actualizamos todos de un solo golpe  
    self.write({  
        'grade': 70.0,  
        'status': 'pass'  
    })
```

Tip de experto: Nunca uses write dentro de un bucle for si puedes evitarlo. Es más eficiente llamar a write una vez sobre todo el conjunto de registros que llamar muchas veces a la base de datos.

3. El método browse(): Se utiliza para obtener un objeto (recordset) a partir de un ID o una lista de IDs conocidos. No consulta la base de datos inmediatamente (es lazy), solo prepara el objeto.

Python

```
def consultar_estudiante(self, id_estudiante):  
    # Convertimos un ID numérico en un objeto de Odoo  
    estudiante = self.env['res.partner'].browse(id_estudiante)  
  
    if estudiante.exists():  
        print(f"El nombre es: {estudiante.name}")
```

4. El método search() y search_count(): Para buscar registros que cumplan ciertas condiciones (dominios).

Python

```
def buscar_reprobados(self):  
    # Buscamos inscripciones con nota menor a 70  
    dominio = [('grade', '<', 70)]  
    # Devuelve el conjunto de registros  
    estudiantes_mal = self.search(dominio)  
    # Devuelve solo el número (más rápido si no necesitas los datos)  
    cantidad = self.search_count(dominio)  
    return estudiantes_mal
```

5. El método unlink(): Se usa para eliminar registros. ¡Cuidado! En Odoo es preferible archivar (active=False) que eliminar, para no perder la trazabilidad.

Python

```
def eliminar_borradores(self):  
    # Buscamos los registros en estado borrador  
    borradores = self.search([('status', '=', 'draft')])  
    # Los eliminamos de la base de datos  
    return borradores.unlink()
```

Ejemplo:

Escenario: Procesar el Cierre de un Curso

Imagina que tenemos un método que, al ejecutarse, debe buscar a todos los estudiantes de un curso, actualizar sus notas y crear una "Inscripción" nueva para el siguiente nivel.

Python

```
def action_close_course_and_upgrade(self):  
    """  
    Este método se ejecuta sobre el modelo 'music.course'  
    Objetivo: Aprobar alumnos y crearles un nuevo registro.  
    """  
    # 1. USO DE SEARCH: Buscamos inscripciones activas de este curso  
    enrollments = self.env['music.enrollment'].search([  
        ('course_id', '=', self.id),  
        ('status', '=', 'pass')  
    ])  
  
    for enrollment in enrollments:  
        # 2. USO DE WRITE: Actualizamos un campo en el registro existente  
        # No usamos 'self.write' porque estamos modificando OTRO modelo (enrollment)  
        enrollment.write({  
            'grade': 100.0,  
            'note': 'Curso completado con éxito'  
        })  
  
        # 3. USO DE CREATE: Creamos un registro en un modelo distinto  
        # Imaginamos que el curso tiene un campo 'next_course_id'  
        if self.next_course_id:  
            self.env['music.enrollment'].create({  
                'student_id': enrollment.student_id.id,  
                'course_id': self.next_course_id.id,  
                'date': fields.Date.today(),
```

```
        'status': 'draft'
    })
```

4. USO DE WRITE EN EL MODELO ACTUAL: Marcamos el curso como cerrado
self.write({'state': 'closed'})

Clases en Odoo

Models

En Odoo, un modelo/tabla es una clase de Python. Para que el framework lo reconozca, debe heredar de `models.Model`. Como buena práctica, cada clase de python debe estar en su propio archivo .py dentro de la carpeta `models`.

```
from odoo import models, fields
class LibraryBook(models.Model):
    _name = 'library.book' # Nombre técnico (será la tabla en la DB)
    _description = 'Modelo para gestionar libros'

    name = fields.Char(string='Título', required=True)
    author_id = fields.Many2one('library.author', string='Autor')
    date_publication = fields.Date(string='Fecha de Publicación')
    pages = fields.Integer(string='Número de páginas')
    active = fields.Boolean(default=True)
```

Este tipo de clase, es el común para modelos base. Pero existen otros tipos que se suelen utilizar según el caso requerido

Transient Model

Se usan para los famosos **Wizards** (ventanas emergentes) estos son modelos transitorios, los cuales cada cierto tiempo se limpia la información de estas tablas, al momento de usarlas se deben asegurar que el objetivo se cumpla y se guarde la información a los modelos que se requieran. Por ejemplo, si quieres un botón que abra una ventana para preguntar "¿A qué usuario quieres transferir este libro?", usarías un `TransientModel`.

Abstract Model

Sirven para definir campos y métodos que quieres que muchos otros modelos compartan, pero no quieres que el modelo abstracto sea una tabla por sí mismo. Pueden ser un poco confusos

al principio porque son "modelos fantasma": existen en el código, pero **no existen en la base de datos**.

- **Ejemplo real:** Odoo usa modelos abstractos para los reportes PDF o para funcionalidades que se "mezclan" con otros modelos.

Campos relacionales

Las relaciones son la unión de diferentes tablas/modelos en Odoo. En lugar de duplicar información, conectamos registros entre sí para crear una red de datos inteligente. Odoo 18 maneja tres tipos principales de relaciones, cada una diseñada para un escenario específico:

1. Many2one (Muchos a Uno)

Es la relación más común. Muchos registros de un modelo apuntan a **un solo** registro de otro modelo.

- **Ejemplo:** Muchos **Libros** pertenecen a un mismo **Autor**.
- **En la base de datos:** Crea una columna con el ID del registro relacionado (una llave foránea).
- # campo = fields.Many2one('modelo.destino', string='Etiqueta')

2. One2many (Uno a Muchos)

Es el reverso del Many2one. Se usa para ver todos los registros relacionados desde el "lado del uno".

- **Ejemplo:** Desde el modelo **Autor**, quieres ver la lista de todos sus **Libros**.
- **Nota importante:** No crea una columna real en la tabla del autor; es solo una "ventana" virtual que busca los Many2one que le pertenecen.
- # campo = fields.One2many('modelo.destino', 'campo_relacion_en_destino', string='Etiqueta')

3. Many2many (Muchos a Muchos)

Varios registros de un modelo se relacionan con varios registros de otro.

- **Ejemplo:** Un **libro** puede tener varios **Tags** (Ficción, Historia), y un **Tag** puede estar en muchos **Libros**.
- **En la base de datos:** Odoo crea automáticamente una "tabla intermedia" para guardar estas conexiones.
- # campo = fields.Many2many('modelo.destino', string='Etiqueta')

Herencia de Modelos (Lógica)

La herencia es uno de los conceptos más potentes de Odoo, ya que te permite modificar o extender funcionalidades existentes (tanto del núcleo de Odoo como de otros módulos) sin tocar el código original. Esto es vital para mantener la escalabilidad. Existen **tres tipos de herencia a nivel de modelos** y **un método principal para las vistas**. Vamos a desglosarlos:

A. Extensión de Clase (`_inherit`)

Es la más utilizada. Se usa para añadir campos o métodos a un modelo que ya existe.

- **Comportamiento:** No crea una tabla nueva. Modifica la tabla existente en PostgreSQL.
- **Uso:** Si quieres añadir el campo "Nacionalidad" al modelo de Clientes (res.partner).

Python

```
class ResPartner(models.Model):  
    _inherit = 'res.partner' # Mismo nombre técnico  
    nationality = fields.Char(string="Nacionalidad")
```

B. Herencia por Prototipo (`_inherit` + `_name`)

Crea un modelo nuevo basado en uno existente, copiando todos sus campos y métodos, pero en una **tabla distinta**.

- **Comportamiento:** Crea una tabla nueva en la DB. Los datos del modelo original no se ven en el nuevo.
- **Uso:** Crear un modelo biblioteca.socio que copie todo de res.partner pero funcione de forma independiente.

Python

```
class LibraryMember(models.Model):  
    _name = 'library.member' # Nuevo nombre técnico  
    _inherit = 'res.partner' # De quién copia la estructura  
    membership_number = fields.Char(string="Número de Socio")
```

C. Herencia por Delegación (`_inherits` - con "s")

Es similar a la composición. El nuevo modelo "contiene" al anterior.

- **Comportamiento:** Crea una tabla nueva, pero vincula los registros mediante un campo Many2one. Si intentas acceder a un campo del modelo padre desde el hijo, Odoo lo trae automáticamente.
- **Uso:** El modelo res.users (Usuarios) hereda por delegación de res.partner. Un usuario es un socio, pero con datos extra de login.

Python

```
class LibraryStaff(models.Model):
    _name = 'library.staff'
    _inherits = {'res.partner': 'partner_id'} # { 'modelo.padre': 'campo_relacion' }
    partner_id = fields.Many2one('res.partner', required=True, ondelete='cascade')
    employee_code = fields.Char(string="Código de Empleado")
```

Herencia de Vistas (Interfaz)

Para modificar una vista existente (añadir un botón, esconder un campo, cambiar un color), no se crea una vista nueva desde cero, sino que se hereda de la original usando el atributo inherit_id. Se utiliza el lenguaje **XPath** para decirle a Odoo: "Busca este elemento y haz algo con él".

Ejemplo de Herencia de Vista:

Si queremos añadir el campo nationality (que creamos arriba) en el formulario de contactos:

XML

```
<record id="view_partner_form_inherit" model="ir.ui.view">
    <field name="name">res.partner.form.inherit</field>
    <field name="model">res.partner</field>
    <field name="inherit_id" ref="base.view_partner_form"/> <!-- Referencia a la vista
original -->
    <field name="arch" type="xml">
        <!-- Localizamos un campo existente para posicionar el nuestro -->
        <xpath expr="//field[@name='vat']" position="after">
            <field name="nationality"/>
        </xpath>
    </field>
</record>
```

Posiciones comunes de XPath:

- inside: Lo pone al final, dentro del elemento seleccionado.
- after: Justo después del elemento.
- before: Justo antes del elemento.
- replace: Reemplaza el elemento por el nuevo.
- attributes: Cambia una propiedad (como hacer un campo obligatorio o invisible).

Decoradores

Los decoradores en Odoo 18 son "instrucciones especiales" que colocamos justo encima de tus funciones en Python. Su misión es decirle al servidor cuándo debe ejecutar ese código sin que tú tengas que llamarlo manualmente.

En Odoo, todos los decoradores empiezan con `@api..` Aquí tienes los tres más importantes que usarás el 90% del tiempo:

1. `@api.depends` (El calculador)

Se usa para campos computados. Es decir, campos cuyo valor depende de otros (como un subtotal que depende de cantidad y precio). Cuándo actúa: Se ejecuta cada vez que uno de los campos mencionados entre paréntesis cambia.

Importante: Siempre debe ir de la mano con un campo que tenga el atributo `compute`.

Python

```
total = fields.Float(string="Total", compute="_compute_total", store=True)
```

```
@api.depends('cantidad', 'precio_unitario')
```

```
def _compute_total(self):
```

```
    for record in self:
```

```
        record.total = record.cantidad * record.precio_unitario
```

Nota técnica: Usamos `store=True` para que el valor se guarde en la base de datos. Si no lo pones, Odoo lo calculará "al vuelo" cada vez que abras la vista.

2. @api.onchange (El dinámico)

Se usa para actualizar la interfaz en tiempo real mientras el usuario escribe, antes de guardar el registro. Es puramente visual. Cuándo actúa: En cuanto el usuario cambia el valor de un campo en el formulario.

Limitación: No guarda datos permanentemente, solo "sugiere" valores al usuario.

Python

```
@api.onchange('partner_id')
def _onchange_partner_id(self):
    if self.partner_id:
        self.email = self.partner_id.email
        self.telefono = self.partner_id.phone
```

3. @api.constrains (El guardián)

Se usa para validaciones de seguridad. Si el usuario intenta guardar algo que no cumple tus reglas, este decorador lanza un error y detiene el proceso. Cuándo actúa: Al momento de hacer clic en "Guardar".

Uso: Validar que una fecha de fin no sea anterior a una de inicio, o que un código tenga X caracteres.

Python

```
from odoo.exceptions import ValidationError
```

```
@api.constrains('edad')
def _check_edad(self):
    for record in self:
        if record.edad < 18:
            raise ValidationError("El socio debe ser mayor de edad.")
```

El archivo ir.model.access.csv

Este es el "quién es quién". Es una tabla que otorga permisos de **Lectura, Escritura, Creación y Eliminación** (CRUD) sobre un modelo específico. Se guarda en la carpeta security/ y debe estar declarado en el `__manifest__.py`.

Estructura del archivo:

Fragmento de código

id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink

access_library_book_admin,access.library.book.admin,model_library_book,base.group_system,1,1,1,1

access_library_book_user,access.library.book.user,model_library_book,base.group_user,1,1,0,0

- **id**: Un nombre único para el permiso.
- **model_id:id**: El nombre del modelo anteponiendo model_. Si tu modelo es library.book, aquí pones model_library_book.
- **group_id:id**: El grupo que recibe el permiso. base.group_system es el administrador total.
- **1 o 0**: 1 concede el permiso, 0 lo deniega.

Categorías y Grupos (Roles por funcionalidad):

Para que el sistema sea escalable, no basta con usar los grupos de Odoo (como base.group_system). Tú puedes crear tus propios roles (ej: "Bibliotecario", "Lector"). Esto se hace en un archivo **XML** dentro de la carpeta security/.

Paso A: Crear la Categoría (La carpeta que agrupa los roles)

XML

```
<record id="module_category_library" model="ir.module.category">
  <field name="name">Biblioteca</field>
  <field name="description">Categoría para gestionar niveles de acceso en la biblioteca.</field>
</record>
```

Paso B: Crear los Grupos (Los roles específicos)

XML

```
<record id="group_library_user" model="res.groups">
  <field name="name">Usuario</field>
  <field name="category_id" ref="module_category_library"/>
  <field name="implied_ids" eval="[(4, ref('base.group_user'))]"/>
</record>
```

```
<record id="group_library_manager" model="res.groups">
  <field name="name">Administrador de Biblioteca</field>
  <field name="category_id" ref="module_category_library"/>
```

```
<field name="implied_ids" eval="[(4, ref('group_library_user'))]"/>
</record>
```

Tip de líder: El campo `implied_ids` (herencia de grupos) permite que el "Manager" herede automáticamente todos los permisos del "User".

3. Permisos mediante Métodos (Seguridad Dinámica)

A veces el archivo CSV no es suficiente porque la regla es más compleja (ej: "Solo el creador del registro puede borrarlo"). Para esto usamos lógica en Python.

A. El método `check_access_rights` Puedes sobrescribir este método para validar si un usuario tiene el "derecho" general de entrar a una función.

B. El método `check_access_rule` (Reglas de Registro) Odoo llama a este método para verificar las "Record Rules" (filtros que limitan qué registros ves).

C. Validación manual en funciones (La más común para Juniors) Si tienes un botón de "Aprobar Presupuesto" y solo el jefe puede usarlo, lanzas una excepción:

Python

```
from odoo.exceptions import AccessError
```

```
def action_approve(self):
    if not self.env.user.has_group('mi_modulo.group_library_manager'):
        raise AccessError("Solo el administrador de la biblioteca puede aprobar este proceso.")
    # Lógica de aprobación...
```

4. Reglas de Registro (Record Rules)

Mientras que el CSV dice si puedes leer la tabla, la **Record Rule** dice qué *filas* puedes ver. Por ejemplo, "Un profesor solo puede ver sus propias secciones".

Se definen en XML:

XML

```
<record id="rule_library_book_own" model="ir.rule">
```

```
<field name="name">Solo ver libros propios</field>
<field name="model_id" ref="model_library_book"/>
<field name="domain_force">[(('create_uid', '=', user.id)]</field>
<field name="groups" eval="[(4, ref('group_library_user'))]" />
</record>
```

Resumen final de seguridad en Odoo 18:

1. **CSV**: Control total por modelo (¿Puede entrar a la tabla?).
2. **Grupos/Categorías**: Definen los niveles de cargo (¿Qué rol tiene?).
3. **Record Rules**: Control por registro (¿Puede ver esta fila específica?).
4. **Métodos Python**: Control por acción (¿Puede ejecutar este botón?).

Con esto tienes el mapa completo para dirigir el desarrollo de un módulo sólido en Odoo 18. Como eres nuevo en Python, te recomiendo empezar siempre por el **CSV** y los **Grupos XML**, ya que son declarativos y más fáciles de depurar.

Este material es complementario para completar tu evaluación y desarrollo en odoo. Recomendaciones en el desarrollo de odoo, es que será necesario que reinicies el servicio de docker cada vez que realices un cambio en archivos **.py** y sumar a eso la **actualización** del módulo desde la interfaz de odoo para que reconozca cambios en archivos **.xml** Cuidado que odoo maneja mucho el cacheo de datos, si hay algo que eliminaste o agregaste y verificas tener todo y aún no visualizas el cambio, prueba con un navegador en limpio o borrar caché.

Lo más necesario está en el documento, sin embargo, en algunas cosas tendrás que investigar, asegurate que lo aplicado sea para la versión 18. Cuando veas un error, fíjate mucho en los logs de la consola, pueden indicar el archivo que no pueda reconocer el ORM. Ve odoo como un framework de desarrollo y adáptate a él.

Inicio de evaluación:

Crea un repositorio en github y sube el proyecto una vez lo descargues, deja la rama main con la base del proyecto y crea una rama de desarrollo donde se recomienda subir avances de forma progresiva.

El primer commit debería ser cuando inicies en vacío

El segundo cuando tengas el entorno y ya lo hayas inicializado

Recomendación del tercero cuando tengas la primera fase lista (puede ser más de un pull)

Y el último cuando termines la última fase.

Estructura del proyecto:

Estructura Base del Proyecto

```
techflow_project/    #Carpeta del proyecto
|
|— docker-compose.yml  # Orquestación de contenedores
|— odoo.conf          # Configuración de Odoo
|— addons/            # Directorio de módulos personalizados
|   |— tech_inventory/ # Tu módulo personalizado
|       |— __init__.py ...
```

Ejecución del Proyecto con docker compose

Docker compose es un archivo de configuración con lenguaje yaml, los comandos a utilizar son los básico para iniciar, actualizar y detener nuestros servicios.

```
# Iniciar el proyecto
docker-compose up | docker-compose up -d (para no visualizar logs)

# Ver logs
docker-compose logs -f odoo

# Detener el proyecto
docker-compose down | ctrl + c

# Actualizar módulo después de cambios
docker-compose restart odoo

# Eliminar todo (incluyendo datos)
docker-compose down -v
```

Las versiones actuales de docker no utilizan el (-) guión. Usa los comandos “docker compose” directamente. Si te da errores de permisos solo usa sudo.

Si estás en windows puedes usar la aplicación.

Una vez descomprimido el proyecto, ubicate en la carpeta raíz y ejecuta el comando para iniciar el proyecto, cuando accedas al localhost te debe pedir la configuración de la base de datos, con la configuración inicial en el docker compose y el odoo.conf ya se establece la conexión, y los datos para crearla los pasas por interfaz y Odoo ya establece la conexión. No es necesario ejecutar ningún código sql.

Configuración de tu base de datos - la clave maestra es la que existe en el odoo.conf

Warning, your Odoo database manager is not protected. To secure it, we have generated the following master password for it:

64az-vmud-k3sq

You can change it below but be sure to remember it, it will be asked for future operations on databases.

Master Password:

Database Name:

Email:

Password:

Phone Number:

Language:

Country:

Demo Data: ☒

[Create database](#) [or restore a database](#)

Búsqueda de tu módulo personalizado

Aplicaciones

Q Módulo Buscar...

1-10 / 10

APLICACIONES

Todos

Aplicaciones oficiales

Industrias

CATEGORÍAS

Todos

Ventas

Servicios

Soporte al cliente

Evaluaciones

Contabilidad

Firma electrónica

Inventario

Manufactura

Sitio web

Marketing

Recursos humanos

Productividad

Administración

Localización

Reparar

Información

Centralice, gestione, comparta y haga crecer su biblioteca de información

Más información

Actualizar

TechFlow - Gestión de Activos Tecnológicos

Gestión de activos tecnológicos de hardware

Activar

Información del módulo

Biblioteca de imágenes de Unsplash

Encuentre imágenes gratuitas de alta resolución con Unsplash

Información del módulo

Cobalt Theme

Development, IT development, Design, Tech, Computers, IT, Blogs

Activar

Información del módulo

Graphene Theme

Service, Corporate, Design, Technology, Robotics, Computers, IT, Blogs

Activar

Información del módulo

Paptic Theme

Consultancy, Design, Tech, Computers, IT, Blogs

Activar

Información del módulo

Clean Theme

Legal, Corporate, Business, Tech, Services

Activar

Información del módulo

Buzzy Theme

Corporate, Services, Technology, Shapes, Illustrations

Activar

Información del módulo

Enark Theme

Architect, Corporate, Business, Finance, Services

Activar

Información del módulo

Kee Theme

Technology, Tech, IT, Computers, Stores, Virtual Reality

Activar

Información del módulo

Activa el módulo y ya podrás tener acceso. Aquí mismo tendrás la opción para actualizarlo.

Para ampliar funcionalidades activa modo desarrollador.



Ajustes

Ajustes generales

Usuarios y empresas

Guardar

Descartar

Ajustes



Ajustes generales



Empleados

ID de aplicación

→ Generar una clave de acceso



reCAPTCHA

Proteja sus formularios de spam y abusos.

Herramientas de desarrollador

Activar modo de desarrollador

Activar modo de desarrollador (con activos)

Activar modo de desarrollador (con activos de prueba)

Acerca de



Contexto de la prueba

Vamos a diseñar un **Sistema de Gestión de Inventario de Equipos Tecnológicos**. Imagina que necesitas controlar los equipos que la empresa entrega a sus empleados, hacer seguimiento del costo y disponibilidad de productos.

Actividades:

Valores al Data (archivo existente)

1. Crear un valor pre cargado de una nueva categoría de equipos (queda a tu juicio).
2. Agregar tres registros al modelo de equipos, destinados a las categoría faltantes, incluyendo la que acabas de crear.

Reglas de Negocio en los modelos (Python) - tech.equipment

1. El serial del equipo debe tener obligatoriamente al menos 8 caracteres. Si no, debe lanzar un error y no dejar guardar.
2. El campo **tax_value**, será el valor del campo cost más el 15% del valor ingresado.
3. State por defecto se debe crear en “disponible”
4. Si **employee_id** le asigna a un empleado, el estatus debe cambiar automáticamente a “Asignado”,
5. Crear método para cambiar a estatus “En reparación”, otro para “desincorporado” y otro para volver a colocarlo “Disponible”. Cada metodo debe limpiar el registro de **employee_id**

Interfaz (XML) - tech.equipment

1. Si el estatus del equipo está "En Reparación o Desincorporado", el campo de "Empleado Asignado" debe limpiarse automáticamente (python) y dejar el campo **employee_id** en solo lectura (xml).
2. La alerta debe aparecer solo cuando el registro esté en estatus “en reparación” del resto debe permanecer oculta.
3. La vista list quedará compuesta por el nombre, estatus y la categoría.
4. En la vista tipo form, delimitar la creación y edición de categorías. Debe ser solo seleccionable de los registro cargados.
5. Agregar botones en el header:
 - a. Disponible: Debe estar visible cuando el equipo este en los otros estatus diferentes a él, llamar al método correspondiente.
 - b. En reparación: Visible cuando el estatus sea asignado o disponible, llamar al método correspondiente.

- c. Desincorporado: Visible cuando el estatus sea diferente al de él, llamar al método correspondiente.

Seguridad y Roles

Debes configurar el acceso para que:

1. **Rol: Usuario IT:** Puede ver todos los equipos y editarlos, pero **no puede borrarlos ni crearlos**.
2. **Rol: Gerente IT:** Tiene control total (CRUD) y puede gestionar las categorías.
3. El Administrador de Odoo (base.group_system) debe tener acceso total por defecto.

Una vez cumpla con estas asignaciones puede pasar a la siguiente fase. Se recomienda mantener el orden.

Nuevo modelo

1. Crear nuevo modelo 'tech_rating.py' con los campos:
 - a. equipment_id con relación de muchos a uno con el modelo de equipos
 - b. evaluated_by_id con relación de muchos a uno con el modelo de usuarios res.users agregar que se complete por defecto con el usuario que crea el registro.
 - c. rating_date tendrá la fecha de la valoración.
 - d. rating - será un campo selection con valoraciones de malo a Excelente (agregar mínimo 3)
 - e. is_recommended - debe ser booleano y será verdadero cuando la valoración en el campo rating sea 'Excelente'
 - f. active - boolean por defecto true
 - g. name - computado entre el equipo y la fecha
2. Agregar modelo al init interno y al archivo de seguridad para darle permisos y poder tener acceso, puedes iniciar con el base.group_system.
3. Crear archivo de vista con vistas de tipo form y list con todos los campos. El campo is_recommended debe estar solo lectura en la vista form.
4. Agregar al archivo de menú un ítem para la vista del nuevo modelo con padre el ítem de configuraciones. Respeta la secuencia y que se muestre primero que las categorías.
5. En el modelo de equipos agregar campo rating_ids con relación de uno a muchos con el modelo de valoraciones.
6. En la vista de equipos:
 - a. Agregar nueva pestaña de Valoraciones y agregar el campo relacional para ver esos registros.
7. Crea un nuevo rol "**Valorador**" que solo pueda consultar los registros de los equipos y no acceda a las categorías pero sí puede agregar una valoración en el modelo creado.